GE VERNOVA

ADMS

16.25.1

DMS API User Guide

Copyright

Change History

Date	Change Description
Nov 20, 2024	This is a new document for ADMS 16.14.0.

Contents

1. Introduction

The DMS API Server receives RESTful requests from external systems, repackages them, and forwards them as gRPC calls to one or more configured backend ADMS roles: the DMS Server (NetSrv) and/or the DNAF Server (DnafSrv). It is deployable within the GridOS architecture and communicates with ADMS components exclusively over gRPC.

Disclaimer

Important

The endpoints and data models described in this guide are primarily internal integration interfaces designed for GE Vernova ADMS/OMS platform components. They are not intended as a general-purpose public API for third-party or in-house applications.

Before developing custom external integrations, consult your GE Vernova representative to confirm the supported and forward-compatible integration approach. Use of these APIs outside the documented, product-supported scenarios is at the implementer's risk.

1.1. DMS API Server Configuration

The DMS API Server is configured using the `ApiServerConfig.xml` file. In the `ApiServerConfig.xml`, users can modify the following settings based on their deployment needs.

- **Port:** The port this API server will listen on.
- **HostIp:** The IP address the API server will bind to. If empty, it binds to 0.0.0.0; otherwise the value must be a local interface address.
- **DmsGrpcServer:** Comma-separated list of backend gRPC hostnames in priority order (e.g. `server1,server2,server3`).
- **DmsGrpcServerPort:** This is the gRPC service port.
- **DmsGrpcServerDeadlineSeconds:** This specifies the deadline (in seconds) for gRPC requests. A value of 0 typically means no deadline.
- **DmsGrpcHcFailureTolerance:** Consecutive failed health checks before the current endpoint is considered failed (default 4).
- **DmsGrpcHcPeriodSeconds:** The period in seconds to perform grpc service health check. Set the value to zero to disable the health check. Default value is 10 seconds
- **LogFileName:** This specifies the file path for the log file generated by the API server.
- **LogLevel:** This sets the logging level for the Microsoft Web Host.
- **MaxMessagesInLogFile:** This specifies the maximum number of log messages to keep in the log file.
- **MaxReceiveMessageSize:** This sets the maximum size (in Megabytes) of the incoming messages that the API server can handle.

- **MaxSendMessageSize:** This sets the maximum size (in Megabytes) of the outgoing messages that the API server can handle.
- **EnableDetailedErrors:** This determines whether detailed error messages are returned to the clients when an exception occurs in a service method.
- **SecureWebService:** This indicates whether the web service is running on http or https. When it is True, then it is https.
- **SslCertificatePfxFile:** This specifies the file path for the PFX certificate file used for SSL connections.

1.2. Setup

To connect to the DNAF server, the GRPC_SERVICE_PORT must be set in the `NetServerDnafConfig_localhost.txt` file, followed by starting the DMS API server. This enables users to interact with the API server through POST and GET requests.

To start the DMS API server, you can use a shortcut like the following:

```
> D:\Eterra\Distribution\Server\bincore\DmsApiServer.exe /CONFIG
"%IDMS_ROOT%\config\server\ApiServerConfig.xml"
```

1.3. Control Plan Generation

The DMS API Server provides configurable options for executing control plans, offering flexibility in managing device operations. The `API_PLAN_OPERATION_MODE_ID` parameter in the DNAF configuration file specifies the operation mode for control plans, including settings for retries, timeouts, and execution behavior. This parameter must correspond to a predefined operation mode in the DNAF Configuration Editor.

Additionally, the `API_PLAN_OPERATION_MODE_ID` works alongside the `API_PLAN_EXECUTION_MODE` setting, which determines the execution approach for control plans. In semi-automatic mode, plans remain pending until explicitly triggered, while in closed-loop mode, plans are executed automatically upon generation.

The `API_PLAN_LIFE_REAL_TIME` parameter defines the retention period, in hours, for control plans created through the DMS API Server. After the specified number of hours from a plan's creation, the plan is automatically deleted from the system. This setting helps prevent the accumulation of obsolete plans.

Fractional hours are supported. For example:

- `0.5` = 30 minutes
- `0.0833` (≈ 5/60) ≈ 5 minutes

1.4. Security

The DMS API server can be run using standard HTTP or HTTPS. By default, HTTPS is selected for enhanced security. SSL can be configured through the GRPC_SERVICE_SSL_CERT_FILE and GRPC_SERVICE_SSL_CERT_KEY_FILE in the `NetServerDnafConfig_localhost.txt` file.

1.5. Server Role and REST Endpoints Availability

The DMS API Server uses the ordered `DmsGrpcServer` list to probe and select among the configured backend roles. It always targets one or both of the two supported roles: the DMS Server (NetSrv) and the DNAF Server (DnafSrv); if both are configured they are probed in the specified priority order.

- The order of values in `DmsGrpcServer` establishes priority. The first healthy endpoint is treated as the primary.
- Health checking is enabled when `DmsGrpcHcPeriodSeconds > 0`. After `DmsGrpcHcFailureTolerance` consecutive failed health checks the client automatically advances to the next endpoint in the list and cycles when reaching the end.
- Advances only on consecutive failures; higher-priority endpoints are revisited only when rotation naturally wraps after further failures (no proactive failback).

1.5.1. Plan Manager Dependent Endpoints

Plan Manager–dependent endpoints (generate control plan `/Device/GenerateCtrlPlan`, plan status `/Device/ControlPlanStatus`*, implement plan `/Device/ImplementControlPlan`, FSD plan generation `/Fault/GenerateFSDPlan`) exist only when the API Server is connected to the DNAF Server. When connected to the DMS Server they return UNIMPLEMENTED.

1.5.2. DMS Server Dependent Endpoint

The Active Hazard Event Visualization endpoint (`/Device/VisualizeHazardEventArea`) should be routed to the DMS Server (NetSrv) for system-wide visualization updates. HMS can submit hazard event area updates so ADMS clients can show or clear circuit halo visualization for affected network areas the as event state changes.

2. Supported REST API Calls

The following tables list the details of the supported REST API calls. The request examples can be accessed in the OpenAPI UI from development mode.

> **Note**
>
> API versioning is required for all API requests. For the sample request listed below, V1 is used as an example.

2.1. Cmd

2.1.1. Enable/Disable DNAF Applications

HTTP Method	POST
URI	/api/v{version}/Feeder/DnafAppEnable
Description	Enable/Disable DNAF application for various feeders in different stations.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.1.2. Retrieve the Enable/Disable DNAF Applications Status

HTTP Method	POST
URI	/api/v{version}/Feeder/DnafAppStatus
Description	Retrieve the enable/disable DNAF applications status at feeder level.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.1.3. Create Planned Outage

HTTP Method	POST
URI	/api/v{version}/Oms/GeneratePlannedOutage
Description	Create a planned outage for the OMS server.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.1.4. Issue Hazard Management System (HMS) Alarm

HTTP Method	POST
URI	/api/v{version}/IssueHmAlarm
Description	Issues Hazard Management System (HMS) alarms into the ALARM application. Supports two distinct alarm categories: **1. Conflict Manager Alarms** - Detects and reports scheduling conflicts between hazard events and planned work activities: • Hazard Event vs. Switching Request • Hazard Event vs. Planned Outage **2. Event Creation and State Transition Alarms** - Tracks state transitions of hazard events through their lifecycle: • Event Creation (None to Watch) • Watch to Pending • Pending to Active • Active to Complete • Watch/Pending/Active to Cancel

Alarm Message Types	The API generates different alarm messages based on the state transition: • **HM001 (Conflict)**: Scheduled work conflict detected (None to Scheduled) *Example:* Conflict Manager: A scheduled work conflict is detected at BETHPAGE F1043 between Hazard Event RFW-123 and Planned Switch Order SWO-711 from 11/21/2025 17:00 to 11/22/2025 21:00. Review if scheduled work can be postponed; if not, explore alternative switching options. • **HM002 (Creation)**: HMS event created (None to Watch) *Example:* Red Flag Warning event RFW-123 is created in Watch state by user123 • **HM003 (Transition)**: HMS event state transition from one state to another state (e.g., Watch to Pending, Pending to Active) *Examples:* ○ Red Flag Warning event RFW-123 transitioned from Watch to Pending by user123 ○ Red Flag Warning event RFW-123 transitioned from Pending to Active by HMS ○ Red Flag Warning event RFW-123 transitioned from Active to Complete by user123 • **HM004 (Cancel)**: HMS event canceled (e.g., Pending to Cancel, Watch to Cancel, Active to Cancel) *Examples:* ○ Red Flag Warning event RFW-123 is canceled from Watch stage by user123 ○ Red Flag Warning event RFW-123 is canceled from Pending stage by user123 ○ Red Flag Warning event RFW-123 is canceled from Active stage by user123

Input Schema	The request body must be a JSON object with the following fields:
	
	Core HMS Event Information:
	
	• `station` (string, required): Primary station name (e.g., "BETHPAGE")
	• `eventId` (string, required): HMS event identifier (max 28 chars, e.g., "RFW-123")
	• `eventType` (integer, required): Event type enumeration:
	  ◦ `1` = Hazard Event
	  ◦ `2` = Switching Request
	  ◦ `3` = Planned Outage
	  ◦ `4` = Unplanned Outage
	• `eventSubtype` (string, conditional): Event subtype - **Required for state transition alarms only (HM002, HM003, HM004)** (max 28 chars, e.g., "Red Flag Warning", "PSPS Event")
	• `initialState` (integer, required): Previous state (see State Enumeration below)
	• `finalState` (integer, required): New state (see State Enumeration below)
	• `modifiedBy` (string, required): User or system that triggered the change (max 30 chars)
	
	Conflict Information (optional - only for Conflict Manager alarms):
	
	• `conflictInfo` (object, conditional): Required when `finalState` = 1 (hmsScheduled)
	  ◦ `conflictType` (integer, required): Event type enumeration:
	  ▪ `1` = Hazard Event
	  ▪ `2` = Switching Request
	  ▪ `3` = Planned Outage
	  ▪ `4` = Unplanned Outage
	  ◦ `conflictId` (string, required): Conflicting work item ID (max 28 chars, e.g., "SWO-711")
	  ◦ `conflictFeederName` (string, required): Affected feeder with station (max 40 chars, e.g., "BETHPAGE F1043")
	  ◦ `startTime` (string, required): Conflict start time (max 16 chars, format: "MM/DD/YYYY HH:mm")
	  ◦ `endTime` (string, required): Conflict end time (max 16 chars, format: "MM/DD/YYYY HH:mm")
	
	State Enumeration (HmsEventStatus):
	
	• `0` = hmsNone (Initial\unset state)
	• `1` = hmsScheduled (Conflict detected state)
	• `2` = hmsWatch (Watch state)
	• `3` = hmsPending (Pending state)
	• `4` = hmsActive (Active state)
	• `5` = hmsCompleted (Event completed)
	• `6` = hmsCancel (Event canceled)
	
	Valid State Transitions:
	
	• None (0) to Scheduled (1)

Sample Request - Conflict Manager Alarm	{
	"station": "BETHPAGE",
	"eventId": "RFW-123",
	"eventType": 1,
	"eventSubtype": "",
	"initialState": 0,
	"finalState": 1,
	"modifiedBy": "ConflictManager",
	"conflictInfo": {
	"conflictType": 3,
	"conflictId": "SWO-711",
	"conflictFeederName": "BETHPAGE F1043",
	"startTime": "11/21/2025 17:00",
	"endTime": "11/22/2025 21:00"
	}
	}

Sample Request - Event Creation (Watch State)	{
	"station": "BETHPAGE",
	"eventId": "RFW-123",
	"eventType": 1,
	"eventSubtype": "Red Flag Warning",
	"initialState": 0,
	"finalState": 2,
	"modifiedBy": "user123"
	}

Sample Request - State Transition (Watch to Pending)	{
	"station": "BETHPAGE",
	"eventId": "RFW-123",
	"eventType": 1,
	"eventSubtype": "Red Flag Warning",
	"initialState": 2,
	"finalState": 3,
	"modifiedBy": "user123"
	}

Sample Request - Cancellation (Active to Cancel)	{
	"station": "BETHPAGE",
	"eventId": "RFW-123",
	"eventType": 1,
	"eventSubtype": "Red Flag Warning",
	"initialState": 4,
	"finalState": 6,
	"modifiedBy": "Manager"
	}

Sample Response - Success	{
	"alarmIssued": true,
	"logText": ""
	}

Sample Response - Failure	{
	"alarmIssued": false,
	"logText": "Invalid state transition from hmsCompleted to hmsWatch"
	}

Response	Success: • 200 OK: Returns `PbIssueHmAlarmResponse` with `alarmIssued` = true Failure: • 400 Bad Request: Invalid input (e.g., invalid state transition, missing required fields, field length exceeded) • 401 Unauthorized: Authentication required • 404 Not Found: Station not found in DNOM • 500 Internal Server Error: Generic server error
Error Messages	Common validation errors returned in `logText`: • "Substation [name] not found." • "Invalid state transition from [initial] to [final]" • "Field '[fieldName]' exceeds maximum length of [max] characters." • "Field 'event_type' cannot be hmsUnknown (0)." • "Field 'event_subtype' is required for hazard event state transitions alarms." • "Invalid event type for state transition alarm. Only hazard events support state transitions." • "Conflict information: Conflict ID, Conflict Type, Conflict Feeder, Start Time, and End Time must be provided when transitioning to SCHEDULED state."

2.1.5. Create/Update Dynamic Schedule

HTTP Method	POST
URI	/api/v{version}/import/schedule
Description	Create/Update schedule for a device: Schema supports updating schedule [{X}, {Y}] for single device in a substation.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.1.6. Create/Update Single Point of Schedule

HTTP Method	POST
URI	/api/v{version}/import/point_of_schedules
Description	Create/Update schedule for a single point [X, Y]: Schema supports updating single point [X, Y] of a schedule for a list of devices in various substations.

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.1.7. Create/Update Set of Schedules

HTTP Method	POST
URI	/api/v{version}/import/schedule_set
Description	Create/Update list of schedules: The schema supports updating schedules [{X}, {Y}] for a list of devices in multiple substations.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2. Device

2.2.1. Enable/Disable Load Shedding

HTTP Method	POST
URI	/api/v{version}/Device/LoadshedEnable
Description	Set enable/disable load shedding flag for the selected switching devices.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.2. Retrieve the Enable/Disable Load Shedding Status

HTTP Method	GET
URI	/api/v{version}/Device/LoadshedEnable
Description	Retrieve the current load shedding enable/disable flag for the selected switching devices.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.3. Generate a Control Plan with Various Control Types

HTTP Method	POST
URI	/api/v{version}/Device/GenerateCtrlPlan
Description	Generates a control plan for a list of switching devices, supporting multiple control types in a single request.
Input Schema	The request body must be a JSON array of objects. Each object represents a control action for a device. The fields required depend on the control type: • `station` (string): Name of the station containing the device. • `deviceId` (string): Identifier of the device. • `controlType` (string): Type of control to apply. Supported values: `"FastTrip"`, `"Reclosing"`, `"SettingGroup"`, `"GroundTrip"`. • `enableFlag` (boolean, optional): For `"FastTrip"`, `"Reclosing"`, and `"GroundTrip"` control types, indicates whether to enable or disable the control. • `initialValue` (integer, optional): For `"SettingGroup"` control type, the initial setting group index. • `finalValue` (integer, optional): For `"SettingGroup"` control type, the final setting group index.

Sample Request	[
	  {
	  "station": "RAVEN",
	  "deviceId": "22239874",
	  "controlType": "FastTrip",
	  "enableFlag": true
	  },
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22229624",
	  "controlType": "Reclosing",
	  "enableFlag": true
	  },
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22229592",
	  "controlType": "GroundTrip",
	  "enableFlag": false
	  },
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22224761",
	  "controlType": "SettingGroup",
	  "initialValue": 4,
	  "finalValue": 5
	  }
	  ,
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22229592",
	  "controlType": "GroundTrip",
	  "enableFlag": false
	  }
	]

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.4. Implement a Control Plan

HTTP Method	POST
URI	/api/v{version}/Device/ImplementControlPlan
Description	Triggers the execution of a previously generated control plan for a given station and plan ID.

Sample Request	{
	  "station": "BETHPAGE",
	  "planId": "API_EXTERNAL_PLAN_BETHPAGE_0723143318"
	}

Sample Response	{
	  "planSent": true,
	  "logText": "Control plan API_EXTERNAL_PLAN_BETHPAGE_0724091216 sent to plan manager for execution."
	}

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.5. Get the Status of a Control Plan

HTTP Method	GET
URI	/api/v{version}/Device/ControlPlanStatus
Description	Retrieves the status of a control plan for a given station and plan ID.
Sample Request	`GET /api/v1/Device/ControlPlanStatus?stationName=BETHPAGE&planId=API_EXTERNAL_PLAN_BETHPAGE_0724091216`

Sample Response	{
	  "planId": "API_EXTERNAL_PLAN_BETHPAGE_0724091216",
	  "station": "BETHPAGE",
	  "status": "PlanStatusFailed",
	  "controlMovesStatus": [
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22229592",
	  "deviceNetType": 4,
	  "controlType": "Reclosing",
	  "status": "PlanStatusFailed"
	  },
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22229624",
	  "deviceNetType": 4,
	  "controlType": "Reclosing",
	  "status": "PlanStatusFailed"
	  },
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22224761",
	  "deviceNetType": 4,
	  "controlType": "SettingGroup",
	  "status": "PlanStatusCompleted"
	  }
	  ],
	  "logText": ""
	}

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.6. Generate a Control Plan to Enable/Disable Reclosing

HTTP Method	POST
URI	/api/v{version}/Device/GenerateReclosingPlan
Description	Generates a control plan to enable/disable reclosing for the selected switching devices.

Sample Request	[
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22229624",
	  "enableFlag": true
	  },
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22229592",
	  "enableFlag": false
	  },
	  {
	  "station": "SPYGLASS",
	  "deviceId": "286ED4FE-37D8-4A2C-9E73-B14476E28EE2",
	  "enableFlag": true
	  }
	]

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.7. Generate a Control Plan to Issue a Fast Trip Control

HTTP Method	POST
URI	/api/v{version}/Device/GenerateFastTripPlan
Description	Generates a control plan to issue a fast trip control to the selected switching devices.

Sample Request	[
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22224713",
	  "enableFlag": true
	  },
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22224808",
	  "enableFlag": false
	  },
	  {
	  "station": "WINGED FOOT",
	  "deviceId": "AA5641C9-58CB-4825-9A44-933131B6A8F9",
	  "enableFlag": true
	  }
	]

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.8. Generate a Control Plan to Issue a Ground Trip Control

HTTP Method	POST
URI	/api/v{version}/Device/GenerateGroundTripPlan
Description	Generates a control plan to issue a ground trip control to the selected switching devices.

Sample Request	[
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22229592",
	  "enableFlag": false
	  },
	  {
	  "station": "SPYGLASS",
	  "deviceId": "286ED4FE-37D8-4A2C-9E73-B14476E28EE2",
	  "enableFlag": true
	  }
	]

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.9. Generate a Control Plan to Issue a Settings Group Change

HTTP Method	POST
URI	/api/v{version}/Device/GenerateSettingsGroupPlan
Description	Generates a control plan to issue a settings group change to the selected switching devices.

Sample Request	[
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22224761",
	  "initialValue": 4,
	  "finalValue": 5
	  },
	  {
	  "station": "PLUTO",
	  "deviceId": "19749093",
	  "initialValue": 2,
	  "finalValue": 1
	  }
	]

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.10. Get the Control Status

HTTP Method	POST
URI	/api/v{version}/Device/ControlStatus
Description	Retrieves the control status for different control types (FastTrip, Reclosing, SettingGroup, GroundTrip) for the specified switching devices. The response includes the current status for each requested control type and device.
Input Schema	The request body must be a JSON array of objects. Each object should contain: • `station` (string): Name of the station containing the device. • `deviceId` (string): Identifier of the device. • `controlType` (string): Type of control to query. Supported values: `"FastTrip"`, `"Reclosing"`, `"SettingGroup"`, `"GroundTrip"`.

Sample Request	<pre>[
	  {
	  "station": "RAVEN",
	  "deviceId": "22239874",
	  "controlType": "FastTrip"
	  },
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22224713",
	  "controlType": "Reclosing"
	  },
	  {
	  "station": "BETHPAGE",
	  "deviceId": "22224761",
	  "controlType": "SettingGroup"
	  },
	  {
	  "station": "PLUTO",
	  "deviceId": "19749093",
	  "controlType": "SettingGroup"
	  }
	]</pre>

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.11. Get the Fast Trip Enable/Disable Status

HTTP Method	POST
URI	/api/v{version}/Device/FasttripStatus
Description	Retrieves the fast trip enable/disable flag for the specified switching devices. The response contains the current enableFlag for each device.
Input Schema	The request body must be a JSON array of objects. Each object should contain: • `station` (string): Name of the station containing the device. • `deviceId` (string): Identifier of the device.

Sample Request	<pre>[
	  { "station": "BETHPAGE", "deviceId": "22224713" },
	  { "station": "WINGED FOOT", "deviceId": "AA5641C9-58CB-4825-9A44-933131B6A8F9" }
	]</pre>

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.12. Get the Reclosing Enable/Disable Status

HTTP Method	POST
URI	/api/v{version}/Device/ReclosingStatus
Description	Retrieves the reclosing enable/disable flag for the specified switching devices. The response contains the current enableFlag for each device.
Input Schema	The request body must be a JSON array of objects. Each object should contain: • `station` (string): Name of the station containing the device. • `deviceId` (string): Identifier of the device.
Sample Request	`[ { "station": "BETHPAGE", "deviceId": "22229624" }, { "station": "SPYGLASS", "deviceId": "286ED4FE-37D8-4A2C-9E73-B14476E28EE2" } ]`
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.13. Get the Active Protection Setting Index

HTTP Method	POST
URI	/api/v{version}/Device/SettingsGroupIndex
Description	Retrieves the active protection setting group index for the specified switching devices. The response contains the currentValue for each device.
Input Schema	The request body must be a JSON array of objects. Each object should contain: • `station` (string): Name of the station containing the device. • `deviceId` (string): Identifier of the device.

Sample Request	<pre>[
	  { "station": "RAVEN", "deviceId": "3F6A7B2F-B7EC-4A73-8BD8-E56D1BB60011" },
	  { "station": "RAVEN", "deviceId": "B033F5E1-A2DC-4393-BA70-7CEB4D357F7E" }
	]</pre>

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.14. Get the Ground Trip Enable/Disable Status

HTTP Method	POST
URI	/api/v{version}/Device/GroundtripStatus
Description	Retrieves the ground trip enable/disable flag for the specified switching devices. The response contains the current enableFlag for each device.
Input Schema	The request body must be a JSON array of objects. Each object should contain: • `station` (string): Name of the station containing the device. • `deviceId` (string): Identifier of the device.

Sample Request	<pre>[
	  { "station": "BETHPAGE", "deviceId": "22229592" }
	]</pre>

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.15. Get the Coordinates of the Polygons

HTTP Method	POST
URI	/api/v{version}/Device/StationPolygonData
Description	Gets the coordinate pairs of the polygons for given stations.

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.16. Get Devices Contained in the Provided Polygon

HTTP Method	POST
URI	/api/v{version}/Device/GetDevicesInPolygon
Description	Gets devices contained in the provided polygon.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.2.17. Visualize Active Hazard Event Area

HTTP Method	POST
URI	/api/v{version}/Device/VisualizeHazardEventArea
Description	Adds, updates, or clears Active HMS hazard event area data used for circuit halo visualization in the geographic view. Use the same `eventId` with `status = 5` (`hmsCompleted`) to clear an Active event visualization.

Input Schema	The request body must be a JSON object with the following fields:
	• `eventId` (string, required): HMS event identifier.
	• `status` (integer, required): HMS event status.
	  ◦ `0` = hmsNone
	  ◦ `1` = hmsScheduled
	  ◦ `2` = hmsWatch
	  ◦ `3` = hmsPending
	  ◦ `4` = hmsActive
	  ◦ `5` = hmsCompleted (removes the event for the same `eventId`)
	  ◦ `6` = hmsCancel (stored as an event state; not auto-removed)
	• `polygons` (array, optional): Hazard area polygon definitions.
	  ◦ `pts` (array of `{x,y}` points, required when polygon is provided)
	  ◦ `crossedStations` (array of strings, optional): Station names crossed by the polygon. If omitted, server-side logic derives affected stations from polygon geometry.
	• `feeders` (array, optional): Full feeder targets.
	  ◦ `station` (string, required)
	  ◦ `id` (string, required)
	• `feederSections` (array, optional): Partial feeder targets.
	  ◦ `station1` (string, required)
	  ◦ `dev1` (string, required): Starting device for the feeder section.
	  ◦ `station2` (string, optional; defaults to `station1` when omitted)
	  ◦ `dev2` (string, optional; if omitted, section extends to feeder end)
	At least one item in the `polygons`, `feeders`, or `feederSections` area inputs must be provided.

Validation and Processing Workflow	The server processes the request in the following order:
	• Validates `eventId`, `status`, and that at least one area input (`polygons`, `feeders`, or `feederSections`) is present.
	• Validates referenced stations/feeders/devices exist in the current ADMS network model.
	• Persists HMS event data using `eventId` as the key:
	  ◦ `hmsCompleted` removes the event.
	  ◦ Other statuses add or update the event.
	• If there is an effective change, recalculates affected devices and notifies subscribed clients to refresh hazard visualization.

Device Resolution Workflow	After an accepted change, the affected-device set is rebuilt for that event as the union of: • all devices from configured `feeders` • devices located within `polygons` • devices traced from `feederSections` Device IDs are de-duplicated per station before being stored and distributed. Polygon-specific behavior: • Polygons with fewer than 3 points are ignored during device-resolution processing. • If `crossedStations` is provided, processing is constrained to those stations. • If `crossedStations` is not provided, stations are derived from polygon/substation intersection logic.
Model Restore and Synchronization Workflow	When the server state is restored and HMS event data exists, the server recomputes the affected devices for all HMS events and signals all subscribed clients to reload the hazard visualization data. The HMS event fast-dynamics binary is saved as `hms_events.hpb` under the configured server `DYNAMIC_PATH` (that is, `<DYNAMIC_PATH>\hms_events.hpb`).
Sample Request	`{ "eventId": "PSPS01", "status": 4, "polygons": [{ "pts": [{ "x": 930137.7, "y": 827744.3 }, { "x": 922081.0, "y": 819974.1 }, { "x": 947620.3, "y": 788461.5 }], "crossedStations": ["RAVEN"] }], "feeders": [{ "station": "RAVEN", "id": "F8863" }], "feederSections": [{ "station1": "RAVEN", "dev1": "22233938", "station2": "RAVEN", "dev2": "22233017" }] }`
Response Payload	• `eventId` (string): Echoed event ID when the update is accepted. • `accepted` (boolean): `true` only when the event data changed (added, updated, or removed). • `logText` (string): Outcome or validation message (for example: `Added eventId=⋯`, `Updated eventId=⋯`, `Removed eventId=⋯`, or `No changes applied eventId=⋯`). Business-validation failures are typically returned as HTTP 200 with `accepted = false` and details in `logText`.

Response	Success: • 200 (request processed; inspect `accepted` and `logText` for business outcome) Failure: • 400: Bad request (invalid or missing REST request body) • 401: Unauthorized • 502: Backend service error • 503: Service unavailable

2.2.18. Get Isolation Devices for the Provided Branch

HTTP Method	POST
URI	/api/v{version}/Device/GetBranchIsolationDevices
Description	Gets isolation devices for the given branch ID. The inputs for this post request are, • Station name • Branch Id • And an option to get only SCADA isolation devices. If this option is set to false, all the isolation devices are returned.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.3. DNAF

2.3.1. Execute DNAF Request

HTTP Method	POST
URI	/api/v{version}/Dnaf/Solve
Description	Send DNAF solution requests like PLOS, DPF, PRV, etc.

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.4. Query

2.4.1. Query

HTTP Method	GET
URI	/api/v{version}/Query
Description	Query data from DNOM with GET request.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.4.2. Query

HTTP Method	POST
URI	/api/v{version}/Query/Query
Description	Query data from DNOM with POST request.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.4.3. Query_ex

HTTP Method	POST
URI	/api/v{version}/Query/Query_ex
Description	Query data from DNOM with extended POST request.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.4.4. GET Schedule Query

HTTP Method	GET
URI	/api/v{version}/services/schedules
Description	Get the schedule of a device in group (substation) with [{X}, {Y}].
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.5. Topology

2.5.1. Trace

HTTP Method	POST
URI	/api/v{version}/Topology/Trace
Description	Trace devices that meet the specified criteria for the NTP trace function.

Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.5.2. TraceUpstreamGetCommonProtDevices

HTTP Method	POST
URI	/api/v{version}/Topology/TraceUpstreamGetCommonProtDevices
Description	Given a list of devices, this function traces and returns a set of upstream protective devices common to all devices in the list. If only a single device is provided, it returns the upstream protective devices specific to that device.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error

2.5.3. Get the Latest Topology Change Date Time

HTTP Method	POST
URI	/api/v{version}/Topology/LastTopologyChange
Description	Get the latest topology change date and time for the specified list of stations.
Response	Success: • 200 Failure: • 401: Unauthorized • 404: Not found • 500: Generic Server Error